

CV2: State a recomposition v Jetpack Compose - teória

1 Cieľ cvičenia

Cieľom tohto cvičenia je pochopiť prácu so stavom v Jetpack Compose a princíp recomposition. V rámci cvičenia vytvoríte jednoduchú aplikáciu Flashcard Trainer, ktorá reaguje na interakciu používateľa a dynamicky mení obsah obrazovky.

Na rozdiel od predchádzajúceho cvičenia už aplikácia nebude statická. Používateľ bude môcť meniť obsah obrazovky a aplikácia bude na tieto zmeny automaticky reagovať.

2 State v Jetpack Compose

State predstavuje hodnotu, ktorá sa môže počas behu aplikácie meniť a ovplyvňuje to, čo sa zobrazí na obrazovke.

Ak sa state zmení, Compose automaticky prekreslí časť používateľského rozhrania, ktorá od tejto hodnoty závisí.

Typickým príkladom state je napríklad:

- aktuálna kartička
- či je zobrazená odpoveď
- počet správnych odpovedí

3 Funkcie remember a mutableStateOf

Na uloženie state v Compose sa používa funkcia `remember` spolu s `mutableStateOf`.

```
var showAnswer by remember {  
    mutableStateOf(false)  
}
```

- `remember` zabezpečí, že hodnota zostane zachovaná medzi recompositions.
- `mutableStateOf` zabezpečí, že Compose bude sledovať zmeny tejto hodnoty.

Pri číselných hodnotách je vhodné použiť aj `mutableIntStateOf`.

```
var currentCardIndex by remember {  
    mutableIntStateOf(0)  
}
```

4 Recomposition

Recomposition, alebo inak rekompozícia, je proces, pri ktorom Compose znovu vykreslí časť používateľského rozhrania.

Keď sa zmení state, Compose zistí túto zmenu a prekreslí iba tie composable funkcie, ktoré od danej hodnoty závisia.

To znamená, že nie je potrebné ručne meniť texty na obrazovke ani manuálne obnovovať celé rozhranie. Stačí zmeniť dáta a Compose sa postará o zvyšok.

5 Dátový model

V našom prípade aplikácia pracuje s jednoduchým dátovým modelom kartičky.

```
data class Flashcard(  
    val question: String,  
    val answer: String  
)
```

Každá kartička obsahuje otázku a odpoveď. Z týchto objektov je následne vytvorený zoznam kartičiek.

```
val flashcards = listOf(  
    Flashcard(  
        question = "What is Jetpack Compose?",  
        answer = "It is a modern way to create Android UI  
                directly in Kotlin."  
    ),  
    Flashcard(  
        question = "Do I need XML with Compose?",  
        answer = "No."  
    ),  
    Flashcard(  
        question = "What is state?",  
        answer = "It is just data the UI remembers and reacts to  
                when it changes."  
    )  
)
```

6 Ako sa mení otázka na obrazovke

Toto je najdôležitejšia časť celého cvičenia. Aplikácia neprepína otázky tak, že by ručne prepisovala text na obrazovke. Namiesto toho si drží informáciu o tom, ktorá kartačka je aktuálne zobrazená.

Na tento účel sa používa index aktuálnej kartačky:

```
var currentCardIndex by remember {  
    mutableIntStateOf(0)  
}
```

Tento index určuje pozíciu v zozname kartačiek. Na základe neho sa vyberie aktuálna kartačka:

```
val currentCard = flashcards[currentCardIndex]
```

Používateľské rozhranie potom nezobrazuje pevne zapísaný text, ale hodnoty z objektu `currentCard`.

```
Text(  
    text = currentCard.question  
)
```

Ak sa zmení hodnota `currentCardIndex`, Compose znovu vyhodnotí výraz

```
flashcards[currentCardIndex]
```

a do premennej `currentCard` sa vyberie iná kartačka. Následne sa prekreslí tá časť obrazovky, ktorá zobrazuje otázku a odpoveď.

To je presne mechanizmus, vďaka ktorému sa na obrazovke zobrazí ďalšia otázka bez toho, aby bolo potrebné ručne meniť text.

7 Zobrazenie odpovede

Okrem zmeny kartačky aplikácia pracuje aj so stavom, ktorý určuje, či sa má zobraziť odpoveď.

```
var showAnswer by remember {  
    mutableStateOf(false)  
}
```

Na začiatku je hodnota `false`, takže sa odpoveď nezobrazí. Po kliknutí na tlačidlo sa hodnota zmení:

```
showAnswer = true
```

Používateľské rozhranie potom môže reagovať podmienkou:

```
if (showAnswer) {  
    Text(  
        text = currentCard.answer  
    )  
}
```

Takto aplikácia najskôr zobrazí iba otázku a až po interakcii používateľa zobrazí aj odpoveď.

8 Riadenie priebehu aplikácie

Aplikácia Flashcard Trainer je riadená pomocou viacerých state premenných.

```
var currentIndex by remember {  
    mutableIntStateOf(0)  
}  
  
var showAnswer by remember {  
    mutableStateOf(false)  
}  
  
var knownCount by remember {  
    mutableIntStateOf(0)  
}  
  
var unknownCount by remember {  
    mutableIntStateOf(0)  
}  
  
var isFinished by remember {  
    mutableStateOf(false)  
}
```

Každá z týchto premenných ovplyvňuje to, čo sa zobrazí na obrazovke:

- `currentIndex` určuje, ktorá karta sa zobrazuje.
- `showAnswer` určuje, či sa má zobrazíť odpoveď.
- `knownCount` a `unknownCount` uchováajú priebežné výsledky.
- `isFinished` určuje, či sa má ešte zobrazovať tréning alebo už výsledná obrazovka.

9 Prechod na ďalšiu kartu

Po vyhodnotení odpovede sa aplikácia musí presunúť na ďalšiu kartu. To sa dá realizovať pomocnou funkciou:

```

fun moveToNextCard() {
    if (currentCardIndex < flashcards.lastIndex) {
        currentCardIndex++
        showAnswer = false
    } else {
        isFinished = true
    }
}

```

Ak ešte existuje ďalšia kartička, zvýši sa index a zároveň sa nastaví `showAnswer` na `false`, aby sa na novej kartičke opäť najskôr zobrazila iba otázka.

Ak už ďalšia kartička neexistuje, nastaví sa stav dokončenia.

10 Podmienené vykresľovanie

Používateľské rozhranie môže reagovať na state pomocou podmienok.

```

if (isFinished) {
    ResultScreen(...)
} else {
    FlashcardContent(...)
}

```

To znamená, že na základe jednej hodnoty môže aplikácia prepínať medzi dvoma odlišnými časťami rozhrania.

Podobne je možné podmienene zobrazovať odpoveď alebo ovládacie tlačidlá.

11 Tok aplikácie

Používateľ najskôr vidí otázku z aktuálnej kartičky.

Po kliknutí na tlačidlo sa zobrazí odpoveď.

Používateľ následne označí odpoveď ako správnu alebo nesprávnu.

Aplikácia:

- aktualizuje príslušné počítadlo,
- posunie sa na ďalšiu kartičku,
- skryje odpoveď na novej kartičke.

Po spracovaní poslednej kartičky sa zobrazí výsledná obrazovka.

12 Čo si máte z cvičenia odniesť

Po dokončení cvičenia by ste mali rozumieť tomu:

- čo je state,
- ako funguje `remember`,
- ako `Compose` reaguje na zmeny dát,
- ako sa pomocou indexu mení aktuálne zobrazená otázka,
- ako riadiť správanie aplikácie pomocou state.